STAT 431 Final Project Written Report

Alexander Wang, Ricky Gong, Zheer Wang, Ziqi Xu

Introduction

This report provides an overview and analysis of the Metropolis-Hastings (MH) sampling algorithm, a key method in Markov Chain Monte Carlo (MCMC) simulations. We will begin by reviewing the algorithm's basic principles, then explore performance analysis strategies, including the effects of different proposal distributions and convergence diagnostics. The goal is to understand how the MH algorithm can be applied to sample from complex Bayesian posterior distributions, where the target distribution is available only up to proportionality. We will also compare MH to Gibbs sampling, highlighting their differences in approach and application. Finally, we will illustrate the algorithm with practical examples, demonstrating how various proposal distributions impact convergence and acceptance rates.

Review and Summary

1. **Problem**

The overarching problem is that we have some target probability distribution (a complicated Bayesian posterior distribution) which we want to sample from. However, we only know this target distribution up to proportionality. The Metropolis-Hastings algorithm provides a sampling method for us to get a sample of values $(\theta_1, \theta_2, \cdots, \theta_n)$ such that as $n \to \infty$, our sample approaches a true sample from the true target distribution.

2. **Algorithm**

The basic idea of the algorithm is that we break down the generation of the next sample value θ_{t+1} from θ_t into two steps. The first step is to generate a candidate for θ_{t+1}, which we obtain by sampling from a proposal distribution of our choice *which we already know how to sample from*. This proposal distribution depends on the current state of the Markov chain, θ_t. The second step is to determine whether or not to accept the sampled candidate value for θ_{t+1}. To do this, we calculate an acceptance probability, which is determined by:

$$\text{acceptance probability} = \min\left\{1, \frac{p(\theta_{t+1}|\mathbf{y})}{p(\theta_t|\mathbf{y})} \times \frac{q(\theta_t|\theta_{t+1})}{q(\theta_{t+1}|\theta_t)}\right\}$$

where $p(\theta|\mathbf{y})$ is the posterior distribution, $q(\theta_{t+1}|\theta_t)$ is the proposal distribution, and $\mathbf{y}$ is our data. Note that we only need to know the posterior distribution up to proportionality, since by Bayes' Rule,

$$p(\theta|\mathbf{y}) = \frac{f(\mathbf{y}|\theta)\pi(\theta)}{m(\mathbf{y})} = \frac{f(\mathbf{y}|\theta)\pi(\theta)}{\int_{-\infty}^{\infty} f(\mathbf{y}|\theta)\pi(\theta)\, d\theta} \propto f(\mathbf{y}|\theta)\pi(\theta)$$

where the proportionality is with respect to θ, and so we have the ratio

$$\frac{p(\theta_{t+1}|\mathbf{y})}{p(\theta_t|\mathbf{y})} = \frac{f(\mathbf{y}|\theta_{t+1})\pi(\theta_{t+1})/m(\mathbf{y})}{f(\mathbf{y}|\theta_t)\pi(\theta_t)/m(\mathbf{y})} = \frac{f(\mathbf{y}|\theta_{t+1})\pi(\theta_{t+1})}{f(\mathbf{y}|\theta_t)\pi(\theta_t)}.$$

Hence, we only need to know the posterior $p(\theta|\mathbf{y})$ up to proportionality when calculating the acceptance probability. We then choose to accept the candidate value for θ_{t+1} with the above acceptance probability. Otherwise, we assign θ_{t+1} to have the same value as θ_t.

The pseudocode for the Metropolis-Hastings Algorithm is shown below. Note that the pseudocode generalizes the algorithm to the multi-dimensional case, where $\boldsymbol{\theta}$ contains all the parameters and hyperparameters in the model.

The Metropolis-Hastings algorithm is based on the following assumptions:

- The target distribution $p(\boldsymbol{\theta})$ is only known up to a proportionality constant, i.e., $p(\boldsymbol{\theta}) \propto g(\boldsymbol{\theta})$, where $g(\boldsymbol{\theta})$ is a function proportional to $p(\boldsymbol{\theta})$.
- A proposal distribution $q(\boldsymbol{\theta}^*|\boldsymbol{\theta}_{i-1})$ is chosen, based on the current state $\boldsymbol{\theta}_{i-1}$, to generate candidate samples.

The steps of the algorithm are as follows:

1. **Initialization:** Select an initial value $\boldsymbol{\theta}_0$.
2. **Iteration:** For $i = 1, \ldots, m$, repeat:

a. **Candidate Selection:** Draw a candidate $\boldsymbol{\theta}^* \sim q(\boldsymbol{\theta}^*|\boldsymbol{\theta}_{i-1})$, where q is the proposal distribution.

b. **Acceptance Ratio:** Compute the acceptance ratio

$$\alpha = \frac{g(\boldsymbol{\theta}^*)q(\boldsymbol{\theta}_{i-1}|\boldsymbol{\theta}^*)}{g(\boldsymbol{\theta}_{i-1})q(\boldsymbol{\theta}^*|\boldsymbol{\theta}_{i-1})}$$

where $g(\boldsymbol{\theta})$ is proportional to the target distribution $p(\boldsymbol{\theta}) \propto g(\boldsymbol{\theta})$.

c. **Acceptance Step:** Accept or reject the candidate $\boldsymbol{\theta}^*$:

i. If $\alpha \geq 1$, accept $\boldsymbol{\theta}^*$ and set $\boldsymbol{\theta}_i \leftarrow \boldsymbol{\theta}^*$.

ii. If $0 < \alpha < 1$, accept $\boldsymbol{\theta}^*$ with probability α, i.e.,

$$\boldsymbol{\theta}_i = \begin{cases} \boldsymbol{\theta}^* & \text{with probability } \alpha, \\ \boldsymbol{\theta}_{i-1} & \text{with probability } 1-\alpha \end{cases}$$

3. <u>Why is Metropolis-Hastings (MH) a Time-Invariant MCMC Algorithm?</u>

We analyze why Metropolis-Hastings is a time-invariant Markov Chain Monte Carlo (MCMC) method. Each θ_t is a random variable, as its value is obtained by sampling from the proposal distribution. Further, θ_t is conditionally independent of $\theta_0, \cdots, \theta_{t-2}$ given θ_{t-1} because generating θ_t depends only on the value of θ_{t-1}: Either $\theta_t = \theta_{t-1}$ or θ_t is the value sampled from the proposal distribution, which depends only on θ_{t-1}. This makes the sample of values $\theta_0, \theta_1, \theta_2, \cdots$ a Markov chain. Furthermore, this sample is a time-invariant Markov chain because the proposal distribution for θ_t (which is our Markov chain transition kernel) does not depend on t, except through the value of θ_{t-1}.

4. <u>Comparison to Gibbs Sampling</u>

The Metropolis-Hastings algorithm uses a proposal distribution to suggest a new sample point θ_{t+1} from the current state θ_t. This proposal distribution need not be related to the target

distribution. Further, Metropolis-Hastings uses an acceptance probability which depends on *both* the proposal distribution and the target distribution. Hence, calculating the next value in the sample depends on both the proposal distribution and the target distribution.

In contrast, as we learned in class, Gibbs sampling relies only on a full conditional distribution of one variable given all the others, as it assumes that it is *easier to sample* from this full conditional distribution compared to the joint distribution. Gibbs sampling does not make use of any proposal distribution. Further, Gibbs sampling always accepts the next sample value because it is directly drawn from the full conditional distribution.

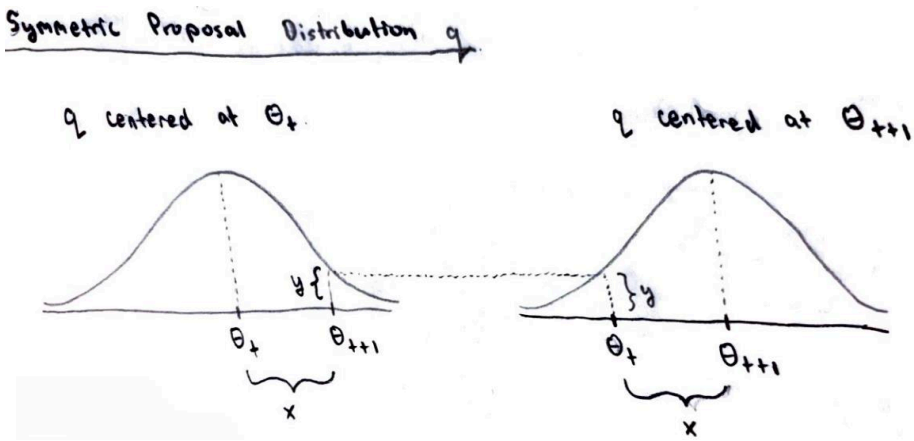
However, Metropolis-Hastings and Gibbs share some similarities. Both are time-invariant MCMC methods; they simply differ in their transition kernel (how we obtain the next θ_t given θ_{t-1}). Additionally, both provide a solution to obtain a sample from a posterior in which it is hard to directly sample from. Thus, while Metropolis-Hastings offers flexibility in choosing the proposal distribution, Gibbs sampling has the advantage of simplicity when conditional distributions are easy to sample. Both methods, however, ultimately aim to sample from a difficult-to-sample posterior distribution.

Illustrations

To illustrate the Metropolis-Hastings (MH) sampling algorithm, we will assess the effect of changes to the proposal distribution on acceptance rates and convergence rates. We demonstrate this by using simulated data. Assume our posterior distribution follows a (standard) exponential distribution $p(\theta|\mathbf{y}) = e^{-\theta}$ up to proportionality. We will compare the effects of using two different types of proposal distributions: symmetric proposal distributions and asymmetric proposal distributions.

1. **Symmetric Proposal Distribution**

When the proposal distribution is symmetric, then $q(\theta_t|\theta_{t+1})/q(\theta_{t+1}|\theta_t) = 1$ for all t since $q(\theta_t|\theta_{t+1}) = q(\theta_{t+1}|\theta_t)$. To see why, consider the picture below:



As we can see, because the proposal distribution q is symmetric, q evaluated at θ_{t+1} when q is centered at θ_t is equal to q evaluated at θ_t when q is centered at θ_{t+1}. In other words, $q(\theta_t|\theta_{t+1}) = q(\theta_{t+1}|\theta_t)$. Thus, the acceptance probability simplifies down to

$$\text{acceptance probability} = \min\left\{1, \frac{p(\theta_{t+1}|\mathbf{y})}{p(\theta_t|\mathbf{y})}\right\}$$

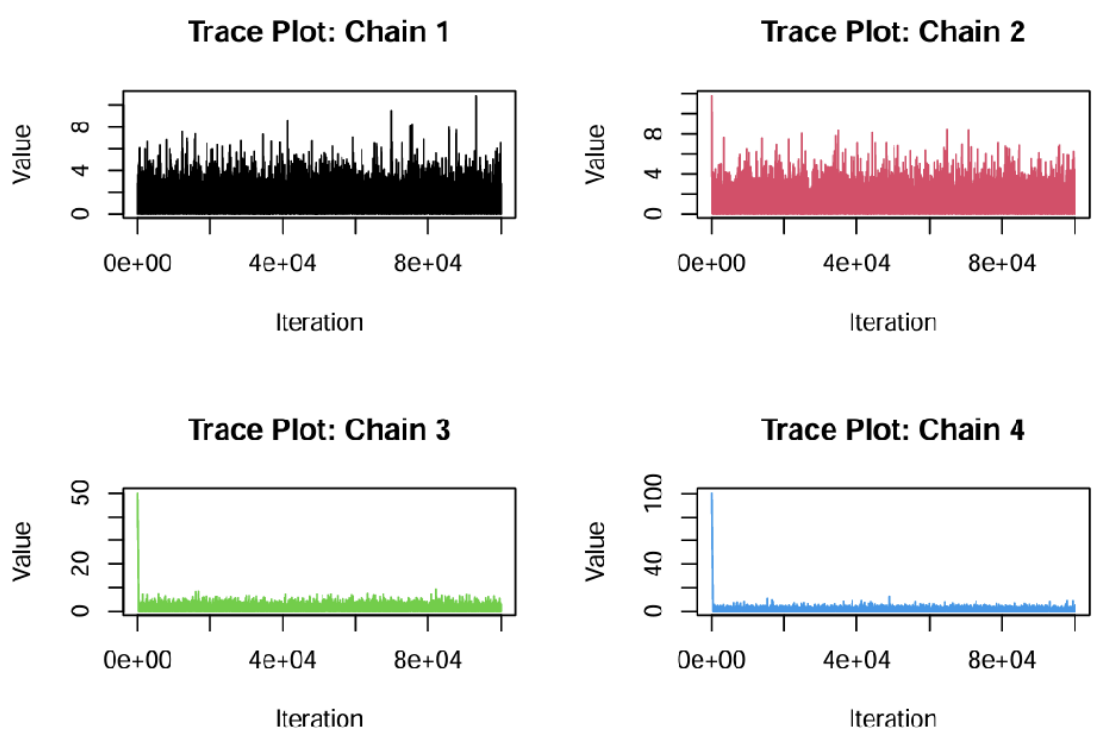
This special case of the MH algorithm is called the Metropolis algorithm. In our simulation, we choose our symmetric proposal distribution to be a Normal distribution with a variance of 1. In other words, $\theta_{t+1}|\theta_t \sim N(\theta_t, 1)$. The code and R output for this is shown below.

```
set.seed(42)

# Target distribution: Exponential distribution
target <- function(x) {
  ifelse(x < 0, 0, exp(-x))
}

# Function to generate samples using the MH algorithm
MH_single_chain <- function(iter = 100000, init = 3, target, proposal_sd = 1) {
  x = numeric(iter)
  x[1] = init
  for (i in 2:iter) {
    current_x = x[i - 1]
    proposed_x = current_x + rnorm(1, mean = 0, sd = proposal_sd)

    A = target(proposed_x) / target(current_x)  # Compute acceptance probability
    if (runif(1) < A) {
      x[i] = proposed_x
```

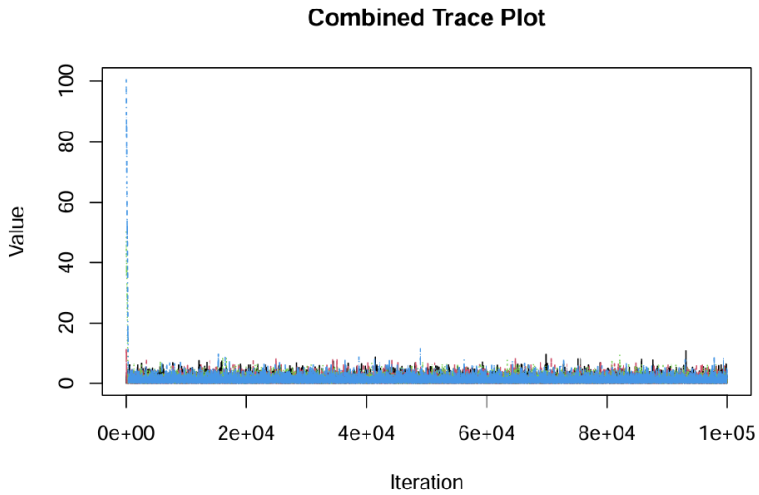
```r
    } else {
      x[i] = current_x
    }
  }
  return(x)
}

# Generate 4 chains with different initial values
chains <- lapply(c(1, 10, 50, 100), function(init) MH_single_chain(init = init, target = target))

# convert to mcmc object
mcmc_chains <- lapply(chains, function(chain) {
  mcmc_chain <- mcmc(chain)
  varnames(mcmc_chain) <- "X"  # Assign variable name
  return(mcmc_chain)
})
```

Diagnostics

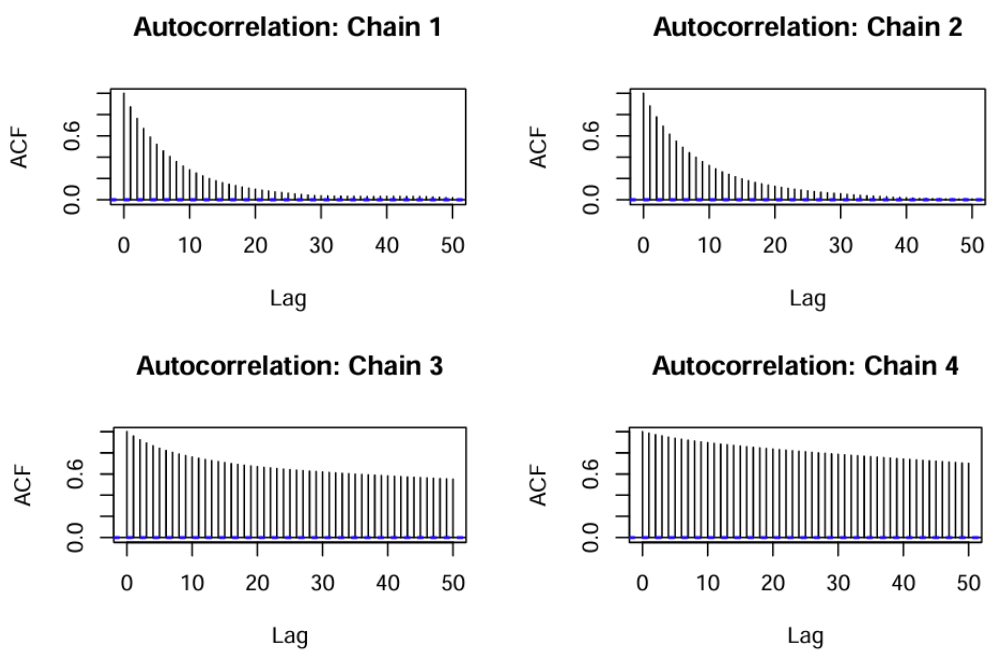
```r
# 1. Trace Plots
par(mfrow = c(2, 2))
for (i in 1:length(mcmc_chains)) {
  traceplot(mcmc_chains[[i]], main = paste("Trace Plot: Chain", i), xlab = "Iteration", ylab = "Value",
}
```



```r
# combined trace plot
par(mfrow = c(1, 1))
combined_chains <- mcmc.list(mcmc_chains)
traceplot(combined_chains, main = "Combined Trace Plot", xlab = "Iteration", ylab = "Value")
```



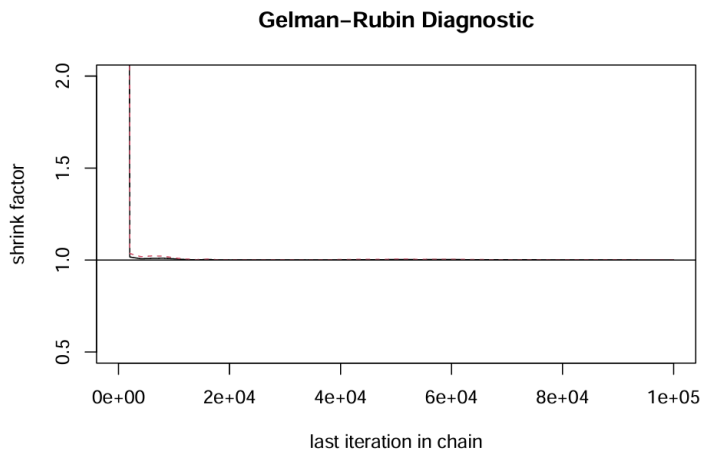

```r
# 2. Autocorrelation Plots
par(mfrow = c(2, 2))
for (i in 1:4) {
  acf(chains[[i]], main = paste("Autocorrelation: Chain", i), lag.max = 50)
}
```



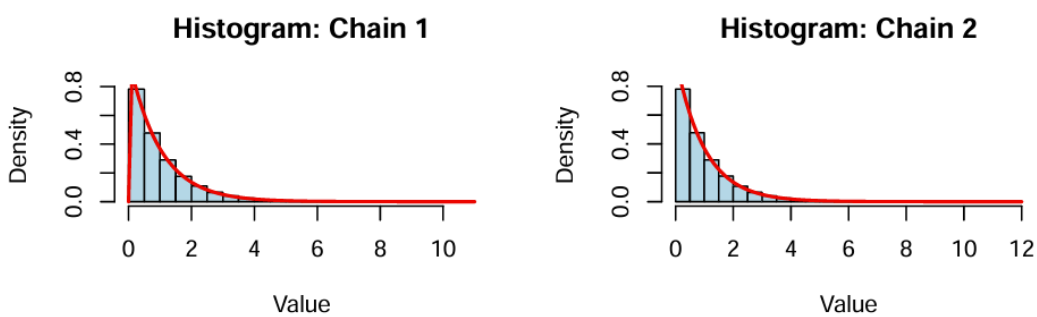
```r
# 3. Gelman-Rubin Diagnostic
gelman_diag <- gelman.diag(mcmc_chains)
print(gelman_diag)
```

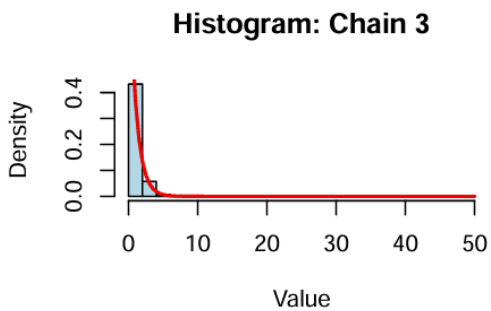
```
## Potential scale reduction factors:
##
##   Point est. Upper C.I.
## X          1          1
```

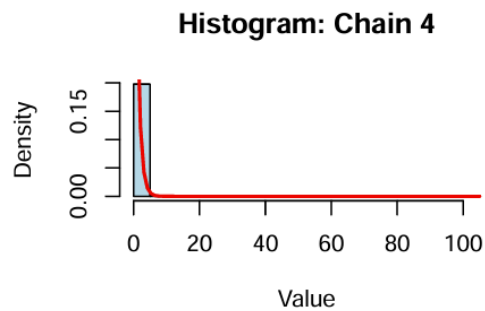
```r
par(mfrow = c(1, 1))
gelman.plot(lapply(chains, mcmc), main = "Gelman-Rubin Diagnostic", ylim = c(0.5, 2))
```




```r
# 4. Histograms of all chains (Empirical Distribution)
par(mfrow = c(2, 2))
for (i in 1:4) {
  hist(chains[[i]], probability = TRUE, breaks = 30,
       main = paste("Histogram: Chain", i), xlab = "Value", col = "lightblue")
  curve(target(x), col = "red", add = TRUE, lwd = 2)
}
```







From the trace plots, we see convergence and fast mixing. Additionally, from the autocorrelation plots, we notice low correlations (and thus high mixing) for chains 1 and 2. For chains 3 and 4, we also see a desired decrease in correlation as the lag increases, although the decrease is not as significant as chains 1 and 2. This is likely because chains 3 and 4 started at more extreme values of 50 and 100. Last but not least, the Gelman-Rubin Diagnostic plot looks good too as the monitored quantity approaches one and stays next to one. From these results, we conclude a symmetric proposal distribution produces relatively high convergence rates. Furthermore, due to the high mixing of these plots, we conclude that a symmetric proposal distribution will lead to a relatively high acceptance rate, which makes sense since accepting the proposed value more frequently allows you to move across the parameter space more quickly, resulting in higher mixing. For future studies, we can repeat this test but use different variances for the proposal distribution. We predict that a smaller proposal width/variance will lead to a lower mixing rate, as the proposed value will always be very close to the previous value due to the small variance.

2. **<u>Asymmetric Proposal Distribution</u>**

Next, we try an asymmetric proposal distribution. When the proposal is asymmetric, then we have the more general form: the Metropolis-Hastings algorithm. Here, the ratio $q(\theta_t|\theta_{t+1})/q(\theta_{t+1}|\theta_t)$, also known as the "Hastings ratio", is included in the acceptance probability calculation. In our simulation, we choose our asymmetric proposal distribution to be a Gamma distribution. In particular, $\theta_{t+1}|\theta_t \sim \mathrm{Gamma}(\alpha, \beta)$ where $\alpha = \kappa\theta_t$ and $\beta = \kappa$. We choose $\kappa = 2$, as κ is an adjustment parameter which controls the spread of the proposal

distribution. Note that choosing these values for α and β allows the mean of our proposal distribution to be θ_t. The code and R output is below.

```r
set.seed(42)

# Target distribution: Exponential distribution
target <- function(x) {
  ifelse(x < 0, 0, exp(-x))
}
```

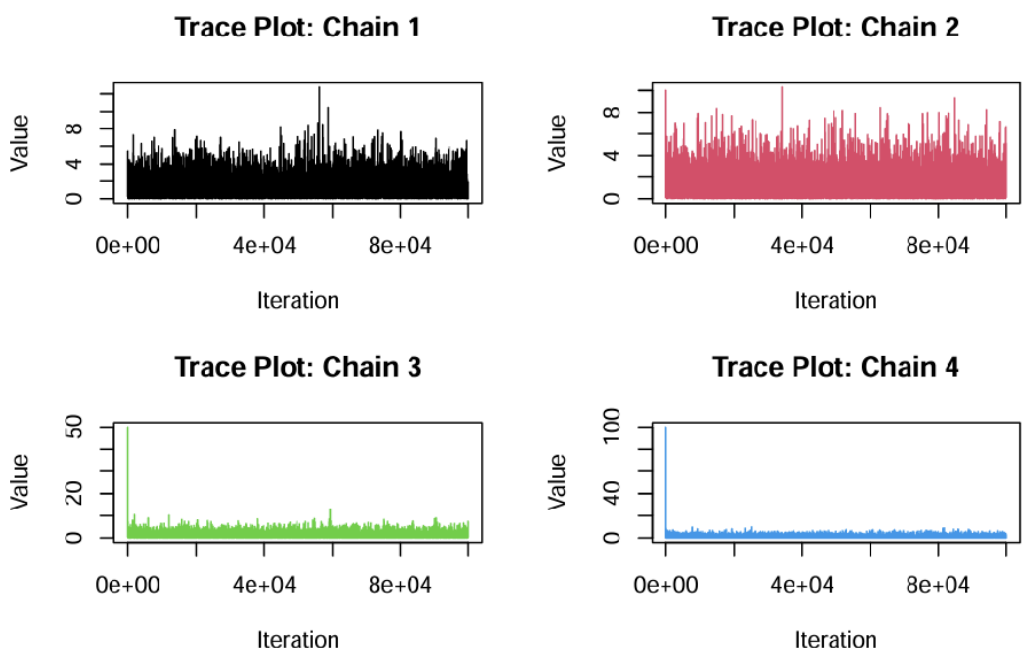
```r
# Metropolis-Hastings function with Gamma proposal distribution
MH_single_chain_gamma <- function(iter = 100000, init = 3, target, kappa = 2) {
  x = numeric(iter)
  x[1] = init

  for (i in 2:iter) {
    current_x = x[i - 1]  # Current state

    # Gamma proposal parameters (based on current_x)
    alpha = kappa * current_x
    beta = kappa

    # Sample proposed value from Gamma distribution
    proposed_x = rgamma(1, shape = alpha, rate = beta)

    # Compute acceptance probability
    q_proposed_given_current = dgamma(proposed_x, shape = alpha, rate = beta)
    q_current_given_proposed = dgamma(current_x, shape = kappa * proposed_x, rate = kappa)

    A = (target(proposed_x) * q_current_given_proposed) /
        (target(current_x) * q_proposed_given_current)
```

```r
    # Accept or reject the proposed value
    if (runif(1) < A) {
      x[i] = proposed_x
    } else {
      x[i] = current_x
    }
  }

  return(x)
}

# Generate 4 chains with overdispersed initial values
chains_gamma <- lapply(c(1, 10, 50, 100), function(init) {
  MH_single_chain_gamma(iter = 100000, init = init, target = target, kappa = 2)
})

# convert to mcmc object
mcmc_chains_gamma <- lapply(chains_gamma, function(chain) {
  mcmc_chain <- mcmc(chain)
```

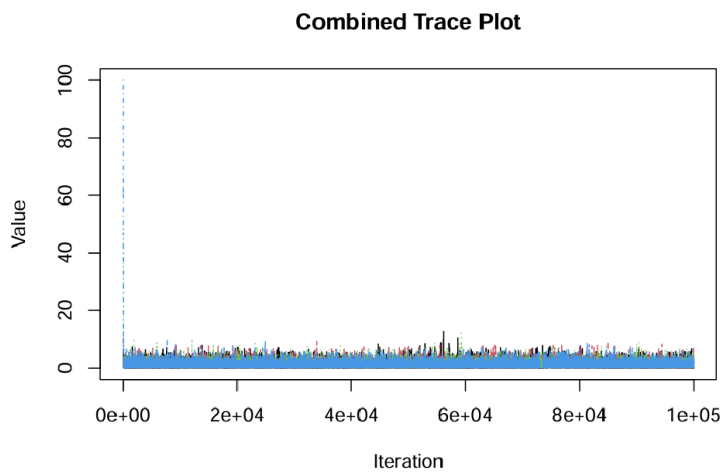
```r
  varnames(mcmc_chain) <- "X"
  return(mcmc_chain)
})
```

Diagnostics

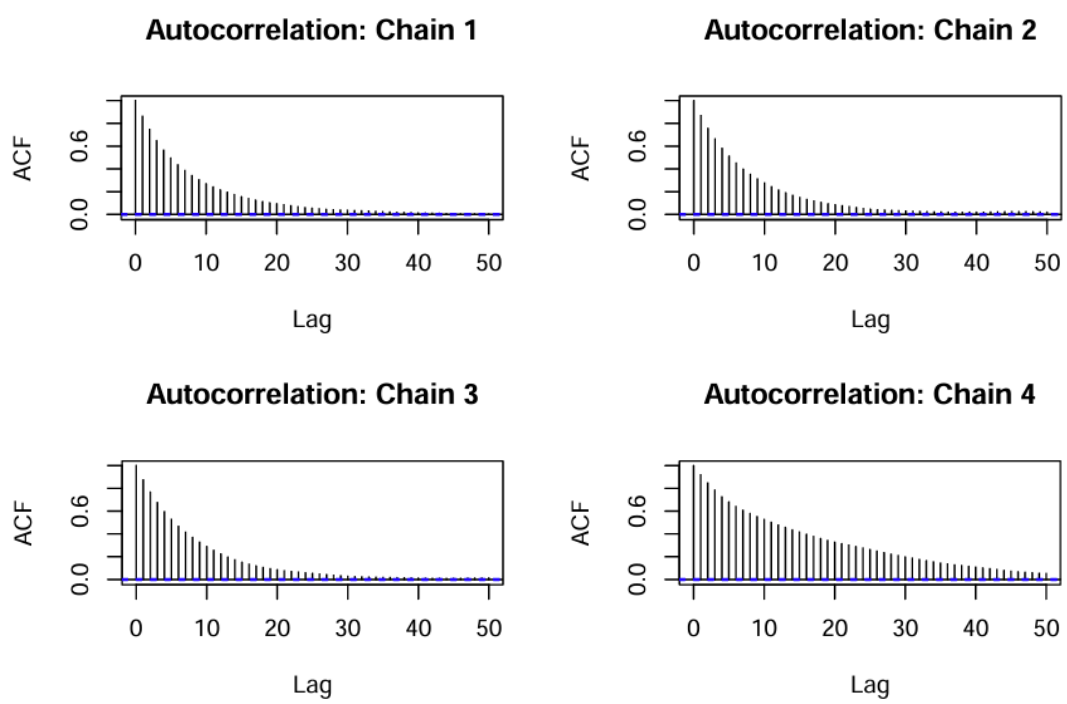
```r
# 1. Trace Plots for individual chains
par(mfrow = c(2, 2))
for (i in 1:length(mcmc_chains_gamma)) {
  traceplot(mcmc_chains_gamma[[i]], main = paste("Trace Plot: Chain", i),
            xlab = "Iteration", ylab = "Value", col = i)
}
```




```r
# Combined Trace Plot
par(mfrow = c(1, 1))
combined_chains_gamma <- mcmc.list(mcmc_chains_gamma)
traceplot(combined_chains_gamma, main = "Combined Trace Plot",
          xlab = "Iteration", ylab = "Value")
```



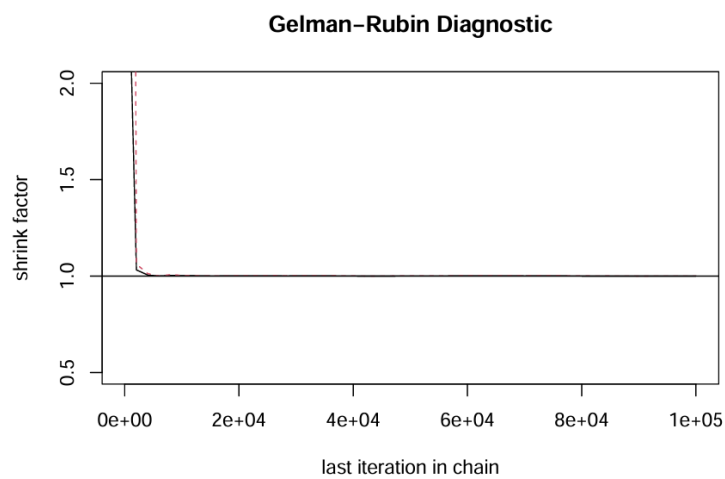

```r
# 2. Autocorrelation Plots for individual chains
par(mfrow = c(2, 2))
for (i in 1:4) {
  acf(as.numeric(chains_gamma[[i]]), main = paste("Autocorrelation: Chain", i), lag.max = 50)
}
```



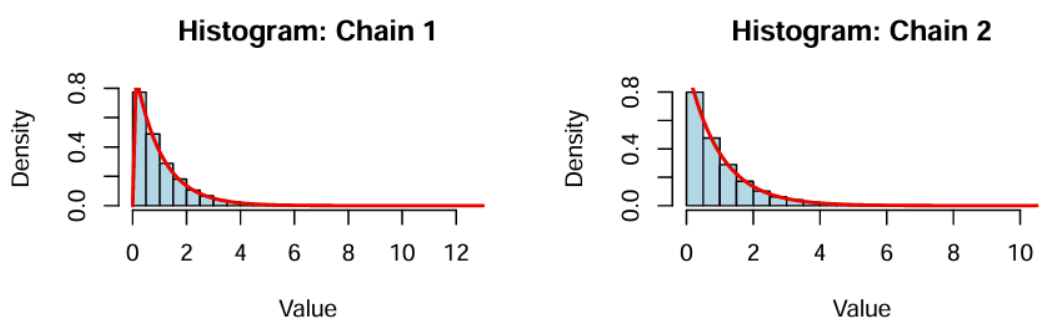
```r
# 3. Gelman-Rubin Diagnostic
gelman_diag_gamma <- gelman.diag(mcmc_chains_gamma)
print(gelman_diag_gamma)
```

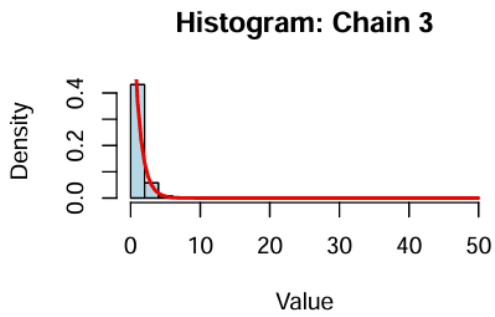
```
## Potential scale reduction factors:
##
##   Point est. Upper C.I.
## X          1          1
```

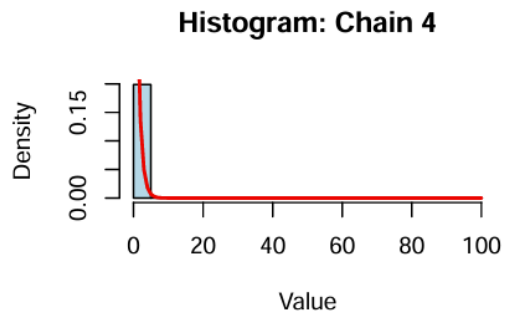
```r
# Gelman-Rubin Plot
par(mfrow = c(1, 1))
gelman.plot(lapply(chains_gamma, mcmc), main = "Gelman-Rubin Diagnostic", ylim = c(0.5, 2))
```




```r
# 4. Histograms of individual chains with target distribution
par(mfrow = c(2, 2))
for (i in 1:4) {
  hist(chains_gamma[[i]], probability = TRUE, breaks = 30,
       main = paste("Histogram: Chain", i), xlab = "Value", col = "lightblue")
  curve(target(x), col = "red", add = TRUE, lwd = 2)  # Overlay target distribution
}
```







From the trace plots, we see convergence and fast mixing. Additionally, from the autocorrelation plots, we notice low correlations (and thus high mixing) for all four chains. Last but not least, the Gelman-Rubin Diagnostic plot looks good too as the monitored quantity approaches one and stays next to one. From these results, we conclude an asymmetric proposal distribution produces relatively high convergence rates. Furthermore, due to the high mixing of these plots, we conclude that we have a relatively high acceptance rate as well. For future studies, we can repeat this test but use different values for κ.

Conclusion

In this report, we explored the Metropolis-Hastings algorithm through practical illustrations and performance analysis. Both symmetric and asymmetric proposal distributions demonstrated high acceptance and convergence rates in the simulations. Our findings suggest that the algorithm performs effectively in exploring the parameter space, leading to reliable sampling from the target distribution. For future work, it would be valuable to test additional types of symmetric and asymmetric proposal distributions to further understand their impact on the performance of the algorithm, and to further see the effects of the changes in proposal distributions, as our results could have been a coincidence for the specific proposal distributions we chose. This could provide more insights into optimizing the algorithm for different types of Bayesian posterior distributions.

Appendix

Alex Wang: Completed the "Review and Summary" portion of the report, as well as writing out the explanations and analyses of the illustrations.

Ricky Gong: Completed all the R code for the "Illustrations" portion of the report.

Zheer Wang: Helped Alex and Ricky with their tasks and resolved all formatting issues.

Ziqi Xu: Helped Alex and Ricky with their tasks and resolved all formatting issues.

References

Machine Learning TV. (2022, February 12). Understanding Metropolis-Hastings algorithm. Youtube. https://www.youtube.com/watch?v=0lpT-yveuIA

Navarro, D. (2023, April 12). The metropolis-hastings algorithm. Notes from a data witch. https://blog.djnavarro.net/posts/2023-04-12_metropolis-hastings/

Plummer, M. (2024). *coda: Output Analysis and Diagnostics for MCMC* (Version 0.19-4.1) [R package]. https://www.rdocumentation.org/packages/coda/versions/0.19-4.1

R Core Team. (n.d.). *The R Project for Statistical Computing*. https://www.r-project.org/

Stephens, M. (2018, April 23). *The Metropolis Hastings Algorithm.* https://stephens999.github.io/fiveMinuteStats/MH_intro.html